

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 2

MATH FOUNDATIONS & EDA

Statistics, Probability, Linear Algebra & Exploratory Data Analysis

The Phase 1→2 transition: the mathematical foundations every ML model rests on — descriptive statistics, probability, and linear algebra — applied immediately through a full Exploratory Data Analysis on a real dataset using Seaborn and correlation analysis.

PHASE 1 → 2 • 40 HOURS • 5 TRAINING DAYS • HANDS-ON

Prepared by BinX Tech
Palestine | Nablus

BinX
TECH

1. Week 2 Overview

Week	Week 2 of 10 — Phase 1 → Phase 2 transition
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 3-4 combined)
Format	On-site / Remote / Hybrid — all work in Jupyter notebooks committed to GitHub
Focus	Descriptive statistics, probability & distributions, linear algebra (vectors, matrices, dot products), and a full EDA on a real dataset (Seaborn, correlation, outliers, data storytelling)
Prerequisite	Week 1 completed: working Python environment, NumPy, Pandas, Matplotlib fluency
Mentor Supervision	Daily check-in; the Day 5 EDA notebook is reviewed as the first graded mini-deliverable milestone

2. Week 2 Learning Objectives

- Compute and interpret descriptive statistics (mean, median, mode, variance, standard deviation) and explain when each measure is appropriate.
- Apply core probability concepts — including conditional probability and Bayes' theorem — and recognize common distributions.
- Explain the linear-algebra objects ML runs on: vectors, matrices, dot products, and matrix multiplication.
- Connect these math foundations directly to how ML models represent data and make predictions.
- Perform a complete Exploratory Data Analysis: univariate and bivariate analysis, correlation, and outlier detection.
- Communicate findings through clear Seaborn visualizations and a written data-storytelling summary.

3. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Descriptive statistics: central tendency, spread, and their meaning for data
Day 2	8 hrs	Probability fundamentals: rules, conditional probability, Bayes' theorem, distributions
Day 3	8 hrs	Linear algebra for ML: vectors, matrices, dot products, matrix multiplication
Day 4	8 hrs	EDA part 1: univariate analysis, distributions, and outlier detection with Seaborn
Day 5	8 hrs	EDA part 2: bivariate analysis, correlation, data storytelling; full EDA notebook

4. Day-by-Day Curriculum

Day 1 — Descriptive Statistics — 8 hours

Learning Objectives

- Compute mean, median, and mode and choose the appropriate one for a given dataset.
- Compute and interpret variance, standard deviation, and IQR.
- Explain how outliers affect each measure differently.

Key Topics

- Why descriptive statistics precedes modeling
- Central tendency: mean, median, mode
- Spread: range, variance, standard deviation, IQR
- Percentiles and quartiles

Lesson Content

1.1 Why Statistics Comes Before Modeling

A machine learning model is, at its core, a summary of patterns in data. If you cannot describe a dataset statistically — its center, its spread, its shape — you cannot judge whether a model's behavior is reasonable. Descriptive statistics is the vocabulary for talking about data precisely, and it underpins every EDA and evaluation step that follows.

1.2 Measures of Central Tendency

Measure	What It Is	When to Use It
Mean	The arithmetic average	Symmetric data with no extreme outliers
Median	The middle value when sorted	Skewed data or data with outliers (more robust)
Mode	The most frequent value	Categorical data, or finding the most common value

The key insight: the mean is pulled toward outliers, while the median is not. On income data, where a few very high earners skew the average, the median is usually the more honest measure of a "typical" value. Choosing the right measure is itself an analytical decision.

```
import numpy as np
data = np.array([10, 12, 12, 13, 100])
print(np.mean(data))      # 29.4 (pulled up by 100)
print(np.median(data))    # 12.0 (unaffected by the outlier)
```

1.3 Measures of Spread

Spread describes how far the data is scattered around its center. Two datasets can share the same mean but behave completely differently depending on their spread.

Measure	Meaning
Range	Maximum minus minimum — simple but very sensitive to outliers

Variance	The average squared distance from the mean
Standard deviation	The square root of variance — spread in the same units as the data
IQR (interquartile range)	The range of the middle 50% of the data (Q3 - Q1), robust to outliers

```
data = np.array([10, 12, 12, 13, 100])
print(np.std(data))      # standard deviation
print(np.var(data))      # variance
q1, q3 = np.percentile(data, [25, 75])
print(q3 - q1)           # interquartile range (IQR)
```

1.4 Percentiles and Quartiles

A percentile marks the value below which a given percentage of the data falls — the 25th percentile (Q1), 50th (the median), and 75th (Q3) split the data into quarters. These are the exact values a box plot visualizes, which is why they are central to the outlier detection covered on Day 4.

Hands-On Lab: Describing a Real Dataset

- Step 1: Load a numeric column from a provided dataset into a NumPy array or Pandas Series.
- Step 2: Compute its mean, median, mode, standard deviation, and IQR.
- Step 3: Identify whether the mean or median better represents a 'typical' value, and justify the choice.
- Step 4: Write a Markdown cell summarizing the column's center and spread in plain language.

Tools Used

NumPy • Pandas • Jupyter Notebook

Day 2 — Probability & Distributions — 8 hours

Learning Objectives

- Apply the complement, addition, and multiplication rules of probability.
- Explain conditional probability and Bayes' theorem and where they appear in ML.
- Recognize the normal, binomial, and uniform distributions.

Key Topics

- Probability basics: favorable over total outcomes
- Core rules: complement, addition, multiplication
- Conditional probability $P(A|B)$
- Bayes' theorem: prior, likelihood, posterior
- Common distributions: normal, binomial, uniform

Lesson Content

2.1 Probability Basics

Probability quantifies uncertainty — and ML models fundamentally output probabilities ("85% likely to churn"). A probability is a number from 0 (impossible) to 1 (certain). The probability of an event is the number of favorable outcomes divided by the total number of possible outcomes, when outcomes are equally likely.

2.2 Core Rules

Rule	Statement	Meaning
Complement	$P(\text{not } A) = 1 - P(A)$	The chance something does NOT happen

Addition	$P(A \text{ or } B) = P(A) + P(B) - P(A \text{ and } B)$	The chance of either event
Multiplication (independent)	$P(A \text{ and } B) = P(A) \times P(B)$	Both happen, when unrelated

2.3 Conditional Probability

Conditional probability, written $P(A | B)$, is the probability of A given that B has already happened. It is the foundation of nearly all predictive modeling: "given these customer features, what is the probability they churn?" is a conditional probability question.

$$\# P(A | B) = P(A \text{ and } B) / P(B)$$

2.4 Bayes' Theorem

Bayes' theorem lets you reverse a conditional probability — computing $P(A | B)$ from $P(B | A)$. It is the mathematical basis of the Naive Bayes classifier and of updating beliefs as new evidence arrives:

$$P(A | B) = (P(B | A) \times P(A)) / P(B)$$

Intuition: it combines what you believed before seeing evidence (the prior, $P(A)$) with how strongly the evidence supports it (the likelihood, $P(B|A)$) to produce an updated belief (the posterior, $P(A|B)$).

2.5 Common Distributions

A probability distribution describes how likely each possible value of a variable is. Three you will meet constantly in ML:

Distribution	Describes	Example
Normal (Gaussian)	Symmetric, bell-shaped data clustered around a mean	Human heights, measurement error
Binomial	Number of successes in a fixed number of yes/no trials	Number of heads in 10 coin flips
Uniform	Every outcome equally likely	A fair die roll

The normal distribution matters most: many statistical methods and ML assumptions rely on data being approximately normal, which is one of the first things EDA checks for.

Hands-On Lab: Probability & Distributions in Code

- Step 1: Simulate 10,000 coin flips with NumPy and confirm the proportion of heads approaches 0.5.
- Step 2: Sample from a normal distribution with `np.random.normal` and plot its histogram to confirm the bell shape.
- Step 3: Given a small worked scenario, compute a conditional probability by hand and verify it with a simulation.
- Step 4: Document each result with a Markdown explanation of what it demonstrates.

Tools Used

NumPy • Matplotlib • Jupyter Notebook

Day 3 — Linear Algebra for ML — 8 hours

Learning Objectives

- Represent data samples as vectors and datasets as matrices.

- Compute a dot product and explain why it is central to model prediction.
- Perform matrix multiplication and reason about resulting shapes.

Key Topics

- Why linear algebra is the language of ML
- Vectors: one sample's features
- Matrices: a full dataset (samples $\times$ features)
- The dot product and how models predict with it
- Matrix multiplication and the shape-matching rule

Lesson Content

3.1 Why Linear Algebra Is the Language of ML

Every dataset in ML is a matrix: rows are samples, columns are features. Every model's parameters are vectors or matrices, and training is a sequence of matrix operations. You do not need to derive proofs, but you must understand what these objects are and how they combine — because that is literally what a model is doing internally.

3.2 Vectors

A vector is an ordered list of numbers — in ML, it usually represents one data sample's features, or one row of data. A customer described by [age, income, tenure] is a 3-dimensional vector.

```
import numpy as np
v = np.array([25, 50000, 3]) # a customer: age, income, tenure
```

3.3 Matrices

A matrix is a 2D grid of numbers — a full dataset, where each row is a sample vector and each column is a feature. Its shape is (rows, columns) = (samples, features).

```
X = np.array([[25, 50000, 3],
              [40, 80000, 10],
              [33, 62000, 5]])
print(X.shape) # (3, 3): 3 samples, 3 features
```

3.4 The Dot Product

The dot product multiplies two vectors element-by-element and sums the result, producing a single number. This is the single most important operation in ML: a linear model's prediction is exactly the dot product of a feature vector and a weight vector, plus a bias.

```
features = np.array([25, 50000, 3])
weights = np.array([0.1, 0.0002, 1.5])
prediction = np.dot(features, weights) # 2.5 + 10 + 4.5 = 17.0
```

This is precisely how linear and logistic regression (Week 3) compute their output: $\text{prediction} = \text{dot}(\text{features}, \text{weights}) + \text{bias}$. Understanding the dot product is understanding how these models actually predict.

3.5 Matrix Multiplication

Matrix multiplication applies the dot product across whole matrices at once, letting a model make predictions for every sample in a dataset in a single operation. The rule: an $(m \times n)$ matrix times an $(n \times p)$ matrix gives an $(m \times p)$ matrix — the inner dimensions must match. This is why shape mismatches are the most common error in ML code, and why understanding shapes is essential.

```
X = np.array([[25, 50000, 3], [40, 80000, 10]]) # (2, 3)
w = np.array([0.1, 0.0002, 1.5]) # (3,)
predictions = X @ w # (2,): one prediction per sample
```

Hands-On Lab: Vectors, Matrices & Predictions

- Step 1: Represent three data samples as a (3 × features) NumPy matrix.
- Step 2: Compute the dot product of one sample vector with a weight vector by hand, then verify it with np.dot.
- Step 3: Use matrix multiplication (@) to produce a prediction for all three samples at once.
- Step 4: Deliberately create a shape-mismatch error, read the message, and explain in Markdown why it occurred and how to fix it.

Tools Used

NumPy • Jupyter Notebook

Day 4 — EDA Part 1: Distributions & Outliers — 8 hours

Learning Objectives

- Explain why EDA is a required first step before modeling.
- Perform univariate analysis using Seaborn histograms, box plots, and count plots.
- Detect outliers using the IQR method and decide how to handle them.

Key Topics

- What EDA is and why it comes first
- Seaborn for statistical visualization
- Univariate analysis: histogram, box plot, count plot, KDE
- Outlier detection with the IQR method

Lesson Content

4.1 What EDA Is and Why It Comes First

Exploratory Data Analysis (EDA) is the systematic process of understanding a dataset before modeling — its structure, distributions, relationships, and problems. It is not optional preliminary work; skipping it is the most common cause of failed ML projects, because a model can only be as good as the understanding behind it. EDA is where you catch the data problems that would otherwise silently corrupt every result downstream.

4.2 Seaborn: Statistical Visualization

Seaborn is built on top of Matplotlib and specializes in statistical plots — it produces attractive, informative charts with far less code, and works directly with Pandas DataFrames. It is the primary EDA visualization tool for this program.

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.histplot(data=df, x="age")
plt.show()
```

4.3 Univariate Analysis

Univariate analysis examines one variable at a time to understand its distribution. The core tools:

Plot	Seaborn Function	Reveals
Histogram	sns.histplot()	The shape of a numeric variable's distribution
Box plot	sns.boxplot()	Median, quartiles, and outliers at a glance
Count plot	sns.countplot()	The frequency of each category in a categorical variable

KDE plot	<code>sns.kdeplot()</code>	A smoothed estimate of the distribution's shape
----------	----------------------------	---

4.4 Outlier Detection with the IQR Method

Outliers are values far outside the normal range — they can be genuine rare events or data-entry errors, and either way they distort statistics and models. The standard IQR rule flags any value below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ as a potential outlier — exactly the values a box plot draws as individual points beyond its whiskers.

```
Q1 = df["income"].quantile(0.25)
Q3 = df["income"].quantile(0.75)
IQR = Q3 - Q1
outliers = df[(df["income"] < Q1 - 1.5*IQR) | (df["income"] > Q3 + 1.5*IQR)]
```

An outlier is a question, not a verdict: investigate whether it is a real value or an error before deciding to keep, cap, or remove it. Never delete outliers silently.

Hands-On Lab: Univariate EDA on a Real Dataset

- Step 1: Load a provided real dataset and produce a histogram for each numeric variable.
- Step 2: Produce a box plot for the key numeric variables and identify any outliers visually.
- Step 3: Apply the IQR method in code to flag outliers in at least one column, and decide (with justification) how to handle them.
- Step 4: Produce a count plot for each categorical variable and note any class imbalance.
- Step 5: Document what each distribution reveals in Markdown cells.

Tools Used

Seaborn • Pandas • Matplotlib • Jupyter Notebook

Day 5 — EDA Part 2: Correlation & Data Storytelling — 8 hours

Learning Objectives

- Perform bivariate analysis with scatter plots and grouped box plots.
- Compute and interpret a correlation matrix and heatmap.
- Assemble a complete, narrated EDA notebook on a real dataset.

Key Topics

- Bivariate analysis: scatter plots, grouped box plots
- Correlation and the correlation heatmap
- Correlation is not causation
- The pairplot for scanning relationships
- Data storytelling: turning analysis into a narrative

Lesson Content

5.1 Bivariate Analysis

Bivariate analysis examines the relationship between two variables — the step where genuine insight usually appears, because ML is fundamentally about relationships between features and a target. A scatter plot reveals the relationship between two numeric variables; grouped box plots reveal how a numeric variable differs across categories.

```
sns.scatterplot(data=df, x="age", y="income")
sns.boxplot(data=df, x="category", y="income")
```


5.2 Correlation

Correlation measures how strongly two numeric variables move together, on a scale from -1 to +1: +1 is a perfect positive relationship, -1 a perfect negative one, and 0 no linear relationship. A correlation heatmap shows every pair of variables' correlations at once — one of the most informative single charts in all of EDA.

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm")
```

Correlation is not causation: two variables moving together does not mean one causes the other. EDA identifies relationships; it never proves cause.

5.3 The Pairplot

`sns.pairplot()` plots every numeric variable against every other in a single grid, with distributions on the diagonal — a fast way to scan an entire dataset's relationships at once, useful early in EDA to decide where to look closer.

```
sns.pairplot(df, hue="target")
```

5.4 Data Storytelling

EDA produces charts, but the deliverable is understanding, communicated clearly. Data storytelling means turning the analysis into a narrative: what the data contains, what patterns and problems were found, and what they imply for modeling. This is the same communication skill emphasized throughout the program — a finding nobody understands is not a finding.

5.5 The Full EDA Notebook

Week 2 closes with a complete EDA notebook that combines everything: descriptive statistics, univariate distributions, outlier handling, bivariate relationships, and a correlation analysis, wrapped in a clear Markdown narrative. This notebook is the template for the EDA stage of every project through to the Phase 3 capstone.

Hands-On Lab: Complete EDA Notebook

- Step 1: Produce scatter plots and grouped box plots for the most important variable pairs in the dataset.
- Step 2: Compute the correlation matrix and visualize it as an annotated heatmap.
- Step 3: Identify the two or three strongest relationships and state what they might mean for a future model.
- Step 4: Assemble the full EDA (statistics, univariate, outliers, bivariate, correlation) into one narrated notebook.
- Step 5: Commit the finished EDA notebook to GitHub with a clear commit message.

Tools Used

Seaborn • Pandas • Matplotlib • Jupyter Notebook • Git & GitHub

5. Week 2 Deliverables

By the end of Week 2, every intern must submit the following to their mentor and GitHub repository:

- A descriptive-statistics notebook computing and interpreting central tendency and spread for a real dataset.
- A probability notebook with simulations for coin flips, a normal distribution, and a conditional-probability check.
- A linear-algebra notebook demonstrating vectors, matrices, the dot product, and matrix multiplication for prediction.
- A univariate EDA notebook with distributions, box plots, and documented outlier handling.
- The complete Week 2 EDA notebook (statistics + univariate + bivariate + correlation) with a data-storytelling narrative.
- All Week 2 notebooks committed to the intern's GitHub repository with clear commit messages.

6. Week 2 Evaluation Criteria

Scored by the assigned mentor at the end of Week 2 using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Understanding of math concepts	Recalls definitions, unclear on application	Explains most concepts and links them to ML	Deep understanding; connects math to model behavior fluently
Statistical & probability correctness	Some calculations or interpretations off	Correct calculations and sound interpretation	Rigorous; chooses the right measure with clear justification
EDA thoroughness	Basic plots, shallow analysis	Thorough univariate + bivariate EDA with clear insights	Comprehensive, insight-driven, catches subtle data issues
Visualization & data storytelling	Unlabeled or unclear plots	Clear, labeled plots with a written narrative	Compelling narrative that drives modeling decisions
Notebook quality & Git workflow	Sporadic commits, weak documentation	Regular commits, clear Markdown narrative	Consistent, descriptive, well-organized, reproducible
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

7. Technical Stack — Week 2

Numerical & Stats	NumPy (statistics, random sampling, linear algebra)
Tabular Data	Pandas (describe, corr, quantile)
Visualization	Matplotlib, Seaborn (histplot, boxplot, heatmap, pairplot)
Environment	Python 3.10+, Jupyter Notebook, Git & GitHub
Datasets	Real tabular datasets (e.g. Titanic, Ames Housing, or a provided CSV)

8. Notes & Best Practices

The math this week is taught for use, not for its own sake: every concept is tied directly to how ML models represent data and predict. The Day 5 EDA notebook is the week's centerpiece and the template for the EDA stage of every later project — invest in making its narrative clear, not just its charts correct.